SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Pradeepa S	**Reg. No.:**	192424318
Date:	30/10/2026	**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. **Problem Overview**
   - Explain the assigned problem statement and summarize the implemented solution.
   - Describe the project objectives and key functionalities.
2. **Repository Organization**
   - Evaluate the repository structure, folder organization, and file naming conventions.
   - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
   - Review the source code for readability, modularity, coding standards, and documentation.
   - Identify strengths and areas for improvement.

4. **Version Control Evaluation**
    - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
    - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
    - Demonstrate that the application functions correctly through appropriate testing.
    - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**
    - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
    - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
    - Summarize the overall code review findings.
    - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.

Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
Problem Analysis & Repository Organization(15%)	/4
Code Quality Review(25%)	/5
Version Control Evaluation(20%)	/4
Testing & Validation(15%)	/4
Documentation & Repository Maintenance(15%)	/4
Review Findings, Recommendations & Presentation(10%)	/4
TOTAL	/25

Question:

Online Course Learning Management System: Develop an e-learning platform where instructors can upload courses and learning materials while students can enroll, attend assessments, track progress, and receive certificates.

1. Problem Overview

Problem Statement

The **Online Course Learning Management System (LMS)** is an e-learning platform designed to provide online education and course management. Instructors can create courses, upload learning materials, and conduct assessments. Students can browse courses, enroll in courses, access learning materials, attend assessments, track their progress, and receive certificates after successful completion.

Implemented Solution

The proposed LMS consists of three main users:

- **Administrator**
- **Instructor**
- **Student**

The system provides separate functionalities for each user and stores course, user, assessment, progress, and certificate information in a database.

Project Objectives

1. To provide an online platform for learning.
2. To allow instructors to create and manage courses.
3. To allow students to enroll in courses.
4. To provide online assessments.
5. To track student learning progress.
6. To generate certificates after course completion.
7. To organize learning materials in a centralized platform.

Key Functionalities

Instructor

- Login/Register
- Create courses
- Upload learning materials
- Update and delete courses
- Create assessments
- View enrolled students
- Monitor student progress

Student

- Register/Login
- View available courses
- Enroll in courses
- Access learning materials
- Attend assessments
- View assessment results
- Track course progress
- Receive certificate

Administrator

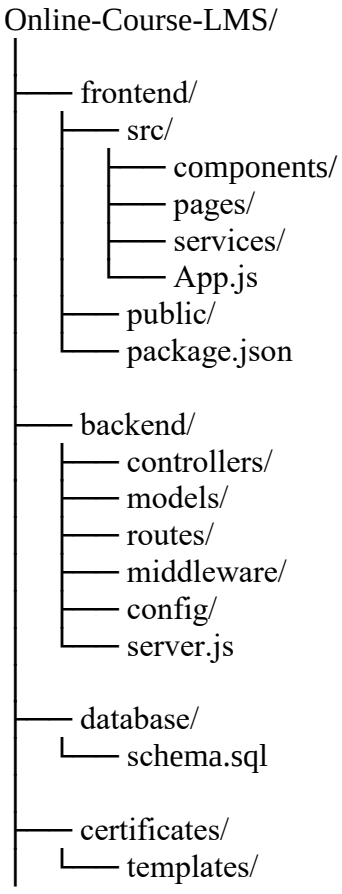
- Manage users
- Manage courses
- Monitor system activities
- Manage assessments and certificates

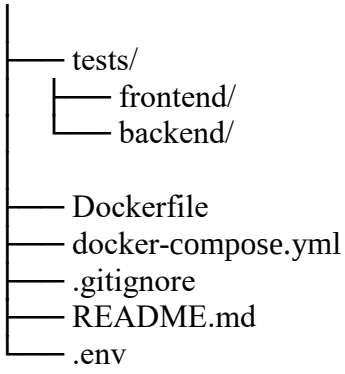
Expected Outcome

The system provides a centralized e-learning environment where instructors can manage educational content and students can learn, complete assessments, monitor their progress, and obtain certificates.

2. Repository Organization

Proposed Repository Structure





Folder Evaluation

Folder/File	**Purpose**
frontend/	Contains the user interface of the LMS
components/	Reusable UI components
pages/	Different application pages
services/	API communication
backend/	Server-side application logic
controllers/	Handles application operations
models/	Defines database entities
routes/	Defines API endpoints
middleware/	Authentication and request processing
database/	Database schema and configuration
certificates/	Certificate templates and generation
tests/	Testing files
Dockerfile	Containerizes the application
docker-compose.yml	Manages application services
.gitignore	Excludes unnecessary files from Git
README.md	Contains project documentation
.env	Stores environment configuration

Maintainability and Collaboration

The repository is divided into separate frontend, backend, database, and testing modules. This modular structure makes the project easier to understand, maintain, debug, and extend.

Standard file and folder naming conventions also help multiple developers work on different modules without confusion.

Screenshot to attach: Repository folder structure in VS Code/GitHub.

3. Code Quality Review

Code Readability

The source code should use meaningful variable, function, class, and file names.

Example:

```
function enrollStudent(studentId, courseId) {
   // Enrollment logic
}
```

The function name clearly indicates its purpose.

Modularity

The application is divided into independent modules such as:

- Authentication
- Course Management
- Student Enrollment
- Assessment
- Progress Tracking
- Certificate Generation

This prevents the entire application logic from being written in a single file.

Coding Standards

The following coding practices are followed:

- Meaningful variable names
- Proper indentation
- Consistent naming conventions
- Reusable functions
- Separation of frontend and backend logic
- Proper error handling
- Comments for important sections
- Avoidance of unnecessary duplicate code

Documentation

Important functions and modules should contain comments explaining their purpose.

Example:

```
// Enrolls a student in the selected course
function enrollStudent(studentId, courseId) {
   // Enrollment implementation
}
```

Strengths

- Modular project structure
- Clear separation of frontend and backend
- Reusable components
- Meaningful naming conventions
- Separate testing directory
- Easy to extend with new LMS features

Areas for Improvement

- Increase automated test coverage.
- Improve error handling.
- Add input validation.
- Add API documentation.
- Improve security for authentication.
- Avoid hard-coded configuration values.
- Add logging for important operations.

4. Version Control Evaluation

Git is used to maintain the source code and track changes made during the development of the LMS.

Branching Strategy



Main Branch

Contains the stable version of the application.

Development Branch

Used for integrating and testing new features.

Feature Branches

Used to independently develop individual functionalities.

Example Commit History

Initial LMS project setup
Add user authentication
Add instructor course management
Add student enrollment
Add assessment module
Add progress tracking
Add certificate generation
Fix authentication validation
Update LMS documentation

Commit Message Evaluation

Good commit messages should clearly describe the change.

Good:

Add student enrollment functionality

Poor:

changes

Meaningful commit messages make it easier to understand the development history and identify changes.

How Git Supports Collaboration

Git supports collaborative development by:

- Maintaining project history.
- Allowing multiple developers to work simultaneously.
- Supporting feature branches.
- Allowing merging of completed features.
- Providing rollback to previous versions.
- Helping resolve conflicts between developers.

Screenshot to attach: GitHub repository branches and commit history.

5. Code Testing and Validation

Testing is performed to verify that all major LMS functionalities work correctly.

Test Cases

Test Case	Input/Action	Expected Result	Status
TC01	Student registration	Student account created	Pass
TC02	Student login	Student dashboard displayed	Pass
TC03	Instructor login	Instructor dashboard displayed	Pass
TC04	Instructor creates course	Course added successfully	Pass
TC05	Instructor uploads material	Material uploaded successfully	Pass
TC06	Student views courses	Available courses displayed	Pass
TC07	Student enrolls in course	Enrollment successful	Pass
TC08	Student attends assessment	Assessment submitted	Pass
TC09	Student views result	Result displayed	Pass
TC10	Student checks progress	Progress displayed	Pass
TC11	Course completion	Certificate generated	Pass

Testing Procedure

1. Open the LMS application.
2. Register a student account.
3. Login using student credentials.
4. Browse available courses.
5. Enroll in a course.
6. Open the learning materials.
7. Attend the assessment.
8. Submit the assessment.
9. Check the result.
10. Track course progress.
11. Complete the required course activities.
12. Verify certificate generation.

Validation

The application is considered successful when:

- Users can login successfully.
- Instructors can create courses.

- Students can enroll in courses.
- Learning materials can be accessed.
- Assessments can be completed.
- Progress is updated correctly.
- Certificates are generated after completion.

Screenshots to attach:

- Login page
- Instructor dashboard
- Course upload page
- Student course enrollment
- Assessment page
- Progress dashboard
- Certificate page

6. Repository Documentation

README

The repository should contain a README.md file with the following information:

Project Title

Online Course Learning Management System

Project Description

The LMS is an e-learning platform that allows instructors to manage courses and learning materials while students can enroll in courses, attend assessments, track their progress, and receive certificates.

Technologies Used

- Frontend: HTML/CSS/JavaScript or React
- Backend: Node.js/Express
- Database: MySQL
- Version Control: Git/GitHub
- Containerization: Docker

Installation

Example:

```
git clone <repository-url>
cd Online-Course-LMS
```

npm install

Running the Application

npm start

Docker Execution

docker compose build
docker compose up

Documentation Evaluation

The documentation should include:

- Project description
- Features
- Technologies used
- Installation instructions
- Usage instructions
- Database information
- API information
- Docker instructions
- Testing instructions

Suggested Improvements

- Add screenshots to README.
- Add architecture diagram.
- Add API documentation.
- Add troubleshooting instructions.
- Add contribution guidelines.
- Include project version information.

The assignment specifically requires evaluation of the README, installation instructions, usage guide, dependencies, and improvements to usability and maintainability.

7. Code Review Findings and Recommendations

Overall Code Review Findings

The Online Course Learning Management System has a modular architecture that separates frontend, backend, database, and testing components. This improves readability and maintainability.

The Git repository provides version control and supports collaborative development through branches and meaningful commits.

The major functional modules include:

- User Authentication
- Course Management
- Student Enrollment
- Learning Materials
- Assessments
- Progress Tracking
- Certificate Generation

Recommendations

Code Quality

- Follow consistent coding standards.
- Use reusable functions and components.
- Add proper error handling.
- Implement input validation.
- Remove duplicate code.

Repository Management

- Use feature branches.
- Use meaningful commit messages.
- Review code before merging.
- Keep the main branch stable.

Testing

- Increase unit testing.
- Add integration testing.
- Test authentication and authorization.
- Automate testing using CI/CD.

Documentation

- Maintain an updated README.
- Add API documentation.
- Include installation and deployment instructions.
- Add screenshots and architecture diagrams.

Future Enhancements

1. AI-based course recommendations.
2. Live online classes.
3. Discussion forums.
4. Personalized learning paths.
5. Mobile application.
6. Advanced student analytics.
7. Automated certificate verification.
8. AI-generated assessments.
9. Notification and reminder system.
10. Learning performance dashboard.

Conclusion

The **Online Course Learning Management System** provides an organized platform for online education. The proposed repository structure improves maintainability, while Git supports version control and collaboration. Proper testing and documentation improve reliability and usability. Future enhancements such as AI-based recommendations, analytics, live classes, and mobile support can further improve the system.